

Índice

Unidad 1 Ingeniería de software y gestión de proyectos	2
1 Software e ingeniería de software	2
2 No Silver Bullet	3
3 Proceso proyecto producto y ciclo de vida	¡Error! Marcador no definido.
4 Modelos de ciclo de vida y criterios de selección	¡Error! Marcador no definido.
5 Componentes y administración del proyecto	¡Error! Marcador no definido.
Unidad 2 Producto requerimientos estimaciones y Scrum	9
6 Agilidad y gestión de producto	¡Error! Marcador no definido.
7 Requerimientos ágiles e historias de usuario	¡Error! Marcador no definido.
8 Estimación y planificación ágil	¡Error! Marcador no definido.
9 Scrum según el material 2026	¡Error! Marcador no definido.
Unidad 3 Gestión de configuración de software	24
10 Fundamentos y conceptos de SCM	¡Error! Marcador no definido.
11 Actividades de SCM y aplicación ágil	¡Error! Marcador no definido.
Fuentes y precisiones para la segunda lectura	¡Error! Marcador no definido.
Bibliografía efectivamente utilizada	¡Error! Marcador no definido.
Material excluido y límites del desarrollo	¡Error! Marcador no definido.
Diferencias relevantes entre fuentes	¡Error! Marcador no definido.
Profundidad que conviene confirmar con la cátedra	¡Error! Marcador no definido.

U1: Ingeniería de software y gestión de proyectos

SOFTWARE E INGENIERÍA DE SOFTWARE

El **software comprende los programas y los elementos asociados** que permiten comprenderlos, construirlos, utilizarlos y mantenerlos, es decir, documentación, datos, procedimientos y reglas. Por eso, entregar software profesional **no consiste únicamente en entregar código ejecutable**, sino también los elementos necesarios para que pueda instalarse, utilizarse, modificarse y mantenerse durante su vida útil.

Se dice que **el software es conocimiento representado en distintos niveles de abstracción**: una misma necesidad puede expresarse con diferente grado de detalle. En un nivel más abstracto se encuentran los requerimientos, que describen qué necesita el sistema; ese conocimiento puede luego precisarse mediante modelos y diseños, hasta llegar a una representación concreta en el código ejecutable. Estas representaciones describen el mismo sistema desde distintos niveles de detalle y deben mantener coherencia entre sí.

Se distinguen tres tipos básicos de software según su propósito:

- **Software de sistema:** Administra recursos y brinda servicios a otros programas;
- **Utilitarios:** Resuelven tareas de apoyo como respaldo o diagnóstico; y
- **Software de aplicación:** orientado a necesidades de los usuarios o del negocio.

CARACTERÍSTICAS DE PRODUCCIÓN DE SOFTWARE

El desarrollo de software es una **actividad intelectual intensiva en personas, donde el esfuerzo principal se concentra en comprender, diseñar y evolucionar la solución**. Por lo que, aunque se construyan dos aplicaciones con necesidades similares, estas no serán idénticas debido a que cambian las reglas, las integraciones, el equipo y el contexto.

A diferencia de los productos físicos, **el software no se desgasta por el uso**. Sin embargo, puede perder adecuación cuando cambia su entorno y volverse más difícil de mantener si acumula modificaciones desordenadas.

El software presenta **menor predictibilidad** porque su desarrollo depende fuertemente de las personas, del conocimiento disponible, de las decisiones tomadas, de los requerimientos y del contexto particular del proyecto. Esta variabilidad también hace que **no exista una producción en masa comparable con la manufactura y que casi ningún producto de software sea exactamente igual a otro**.

Además, no todas las fallas observadas se deben a errores de programación. A diferencia de un producto físico, su estructura no está determinada por un conjunto de leyes físicas universales, sino principalmente por **reglas de negocio, convenciones, decisiones, interfaces y necesidades propias de cada contexto**, que además pueden cambiar con el tiempo.

CRISIS DEL SOFTWARE

La **crisis del software** refiere a las dificultades recurrentes para producir sistemas que satisfagan las necesidades con calidad, dentro de tiempos y costos razonables. Se manifiesta en retrasos, sobrecostos, funcionalidades inadecuadas, defectos, documentación insuficiente y productos difíciles de mantener o extender. Estos problemas no se reducen a errores de programación, sino que también involucran requerimientos, coordinación, decisiones técnicas y administración del proyecto.

Frente a estas dificultades fueron surgiendo distintos enfoques, prácticas y modelos para mejorar la forma de desarrollar y gestionar software. Sin embargo, **el avance tecnológico por sí solo no elimina los problemas fundamentales de la Ingeniería de Software**: sigue siendo necesario comprender qué construir, coordinar el trabajo y comprobar que el producto realmente aporte valor.

INGENIERÍA DE SOFTWARE

La **Ingeniería de Software** aplica un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software.

La construcción de software por **múltiples personas y en múltiples versiones** genera dos desafíos centrales: la **coordinación entre las personas involucradas y la gestión de los cambios** a medida que el software evoluciona.

Las disciplinas de la Ingeniería de Software se organizan en:

- **Técnicas:** transforman necesidades en una solución verificable y utilizable. Por ejemplo, requerimientos, análisis y diseño, construcción, prueba y despliegue.
- **De gestión:** organizan objetivos, actividades, recursos y decisiones, y permiten corregir desvíos. Por ejemplo, planificación, monitoreo y control.
- **De soporte:** acompañan transversalmente el trabajo y ayudan a sostener la integridad y calidad de los resultados. Por ejemplo, gestión de configuración, aseguramiento de calidad y métricas.

El **SWEBOK (Software Engineering Body of Knowledge)** organiza el cuerpo de conocimientos de la Ingeniería de Software y sirve como referencia sobre el alcance de la disciplina; **no es una metodología ni un ciclo de vida**.

El **SEBoK (Systems Engineering Body of Knowledge)** corresponde a la **Ingeniería de Sistemas**, un campo más amplio que considera el sistema completo y las relaciones entre sus componentes, pudiendo incluir software, hardware, personas, procesos y organizaciones.

NO SILVER BULLET

En **No Silver Bullet**, Brooks cuestiona que exista una única innovación tecnológica o de gestión capaz de producir, por sí sola, una mejora radical —del orden de diez veces— en productividad, confiabilidad y simplicidad.

La metáfora de la bala de plata **cuestiona la búsqueda de una solución milagrosa para problemas que tienen distintas causas**. Las herramientas y nuevas formas de trabajar pueden generar mejoras importantes, pero **parte de la dificultad pertenece a la propia naturaleza del software y requiere comprender, diseñar y validar correctamente la solución**.

ESENCIA Y ACCIDENTE

La **esencia es la estructura conceptual del software**: los datos, relaciones, algoritmos, reglas e interacciones que deben concebirse correctamente para resolver el problema. La **dificultad esencial consiste en especificar, diseñar y comprobar esa estructura**.

Los **accidentes son dificultades de producción que no son inherentes a esa estructura**, como representar las ideas en lenguajes incómodos, lidiar con restricciones de memoria, esperar resultados de compilación o coordinar herramientas incompatibles. Una **dificultad accidental puede reducirse o eliminarse sin cambiar la esencia del problema**.

A medida que las herramientas reducen las dificultades accidentales, adquiere mayor peso relativo el trabajo esencial que permanece. Por eso, **acelerar actividades como escribir código no implica necesariamente acelerar en la misma proporción todo el desarrollo ni garantizar que se esté resolviendo correctamente el problema**.

LAS CUATRO DIFICULTADES ESENCIALES

1- COMPLEJIDAD

Un sistema contiene numerosos elementos distintos y muchas relaciones entre ellos. A medida que crece, no sólo incorpora nuevos elementos, sino también **nuevas interacciones y estados posibles**.

Esta complejidad dificulta comprender el conjunto, comunicar decisiones, probar comportamientos y modificar una parte sin afectar otras. Además, se vuelve difícil conservar una visión coherente y gestionar el conocimiento distribuido entre las personas del equipo.

2- CONFORMIDAD

La **conformidad** refiere a las exigencias externas que debe cumplir el software, como reglas, procedimientos e interfaces impuestas por su entorno. Muchas de estas exigencias dependen del contexto y no responden a una regla universal. Rediseñar internamente el programa no elimina la obligación de adaptarse e interactuar con ellas.

3- MUTABILIDAD

El software está sometido a una **presión permanente para cambiar**: los usuarios descubren nuevos usos, cambian las organizaciones y aparecen nuevos entornos técnicos. Además, como el software puede modificarse, muchas adaptaciones del sistema se realizan cambiándolo. Incluso el software exitoso evoluciona, ya que su utilización genera nuevas necesidades y demandas.

4- INVISIBILIDAD

El software no posee una forma espacial natural que permita visualizarlo íntegramente. Existen vistas que representan determinadas partes o aspectos de su estructura, pero combinarlas no produce automáticamente una representación clara del conjunto. Esto dificulta tanto el diseño individual como la comunicación entre personas.

ALCANCE DE LAS MEJORAS TECNOLÓGICAS

Las herramientas y avances tecnológicos pueden producir mejoras importantes al **reducir dificultades accidentales del desarrollo**. Sin embargo, esto no implica que eliminen las **dificultades esenciales** de comprender el problema, definir correctamente qué construir y diseñar una solución adecuada.

Por ejemplo, comprobar que un programa cumple perfectamente una especificación no garantiza que esa especificación represente correctamente la necesidad del usuario.

Trasladando esta idea a la **IA generativa**: una IA puede acelerar considerablemente la producción de código a partir de instrucciones, pero todavía es necesario que las personas definan la intención, las reglas y restricciones del problema, y validen que la solución generada sea correcta y realmente satisfaga la necesidad.

CÓMO ABORDAR LAS DIFICULTADES ESENCIALES

Las dificultades esenciales no pueden eliminarse simplemente incorporando mejores herramientas, pero existen formas de **reducir la incertidumbre y comprender mejor qué solución construir**.

1- Evaluar reutilizar o comprar antes que construir desde cero. Si existe una solución que satisface razonablemente la necesidad, reutilizarla puede evitar resolver nuevamente problemas ya resueltos. La decisión depende de su adecuación al contexto y de las adaptaciones necesarias.

2- Refinar los requerimientos mediante prototipos. Los usuarios no siempre conocen o pueden expresar desde el inicio todo lo que necesitan. Un prototipo permite volver observable una idea, obtener retroalimentación y corregir la comprensión del problema antes de comprometer grandes esfuerzos.

3- Desarrollar incrementalmente. Partir de una solución limitada pero funcional y ampliarla progresivamente permite validar comportamientos, obtener retroalimentación y aprender mientras evoluciona el producto, en lugar de esperar hasta el final para comprobar si la solución funciona como se esperaba.

4- Dar importancia al conocimiento y criterio de las personas. Los procesos, buenas prácticas y herramientas ayudan al desarrollo, pero no reemplazan la capacidad de comprender el problema, tomar decisiones de diseño y construir una solución coherente.

En conclusión, **no existe una única solución para las dificultades del desarrollo de software**. La Ingeniería de Software debe combinar personas, prácticas técnicas y gestión: las herramientas permiten reducir dificultades accidentales, mientras que comprender los requerimientos, diseñar la solución, coordinar el trabajo y gestionar su evolución permite abordar las dificultades esenciales.

RELACIÓN ENTRE PROCESO, PROYECTO, PRODUCTO Y CICLO DE VIDA

PROCESO DE SOFTWARE

Un **proceso de software** es un conjunto estructurado de actividades, métodos, prácticas y transformaciones que las personas utilizan para desarrollar o mantener software y sus productos asociados.

Define qué trabajo debe realizarse y cómo se organiza. Si se quiere administrar el proceso, este debe hacerse explícito, definiendo sus actividades, productos o resultados, roles, responsabilidades y condiciones de realización.

No existe un único proceso adecuado para todos los desarrollos: **su forma depende de la organización, del tipo de sistema y del contexto en el que se trabaja.**

Se identifican cuatro **actividades fundamentales**:

- **Especificación:** definir qué debe producirse y sus restricciones.
- **Desarrollo:** diseñar e implementar la solución.
- **Validación:** comprobar que el software satisface lo requerido.
- **Evolución:** modificarlo frente a nuevas necesidades o cambios.

Estas actividades pueden organizarse y relacionarse de distintas maneras según el proceso y el **ciclo de vida** adoptado.

El **proceso se apoya en tres elementos interdependientes**: procedimientos y métodos; personas con habilidades, formación y motivación; y herramientas y equipos. Las herramientas pueden automatizar y facilitar parte del trabajo, pero **no sustituyen las decisiones de las personas ni garantizan por sí solas que el proceso sea adecuado al contexto.**

PROYECTO Y PRODUCTO

Un **proyecto** es un esfuerzo temporal, con un inicio y un final, orientado al logro de objetivos mediante tareas interrelacionadas que consumen esfuerzo y recursos. Cada proyecto es único: incluso dos proyectos con objetivos similares pueden diferir en equipo, destinatarios, riesgos, presupuesto, calendario o condiciones de ejecución.

El **producto** es la solución que se crea y evoluciona para satisfacer necesidades. Su vida suele superar la de un proyecto, ya que a lo largo del tiempo pueden realizarse distintos proyectos para desarrollarlo, ampliarlo, corregirlo o adaptarlo.

RELACION PROCESO-PROYECTO-PRODUCTO

La relación central es que **el proceso define cómo se organiza el trabajo, el proyecto adapta y aplica ese proceso en un contexto concreto, y mediante ese proyecto se crea o modifica un producto.** Por eso:

- el **proceso** organiza cómo se trabaja;
- el **proyecto** concreta ese trabajo con objetivos, restricciones y recursos;
- el **producto** es aquello que se crea y evoluciona para generar valor y satisfacer necesidades.

CICLO DE VIDA

El **ciclo de vida** es la serie de etapas por las que progresa un producto o proyecto. El **ciclo de vida del producto** abarca desde su concepción hasta su retiro, mientras que el **ciclo de vida del proyecto** abarca desde su inicio hasta su cierre. Un mismo producto puede atravesar distintos proyectos a lo largo de su vida.

Un **modelo de ciclo de vida** representa cómo se organiza el desarrollo a través de sus fases: qué actividades se realizan, en qué orden o relación y cómo se avanza entre ellas. Es una abstracción del proceso y no describe necesariamente todas las responsabilidades, técnicas o herramientas concretas utilizadas.

El ciclo de vida no debe confundirse con una metodología o un framework. Una **metodología organiza métodos y prácticas** para desarrollar el trabajo, mientras que un **framework proporciona una estructura dentro de la cual pueden aplicarse distintas prácticas.**

PROCESOS DEFINIDOS Y EMPÍRICOS

En un **proceso definido** se considera que el trabajo es suficientemente conocido, estable y repetible como para anticipar sus resultados. Por eso, la planificación puede realizarse con mayor detalle y el control consiste principalmente en **comparar lo realizado con lo planificado y corregir los desvíos**. Este enfoque resulta más adecuado cuanto mayor es la estabilidad y el conocimiento previo sobre cómo obtener el resultado.

En un **proceso empírico** se reconoce que existen variables cambiantes y que repetir las mismas actividades no garantiza obtener los mismos resultados. Por eso, el control se basa en **observar lo que realmente ocurre, inspeccionar frecuentemente los resultados y adaptar el trabajo a partir de lo aprendido**. Es especialmente útil cuando existe complejidad, creatividad o incertidumbre.

Esto no significa que un proceso empírico carezca de reglas o planificación. **La diferencia está en cómo se obtiene el control**: el definido se apoya principalmente en la predictibilidad, mientras que el empírico se apoya en la evidencia y la adaptación. Por eso, “definido” no significa simplemente “tener el proceso escrito”.

En los procesos empíricos, el aprendizaje sigue el patrón **asumir → construir → obtener retroalimentación → revisar → adaptar**. Se parte de supuestos, se actúa sobre ellos y los resultados reales permiten confirmarlos o modificarlos.

Definido y empírico describen la lógica utilizada para controlar el proceso; en cambio, secuencial, iterativo o recursivo describen cómo se organiza el avance del desarrollo. Son dimensiones diferentes y no deben confundirse.

MODELOS DE CICLOS DE VIDA Y CRITERIOS DE SELECCIÓN

Los modelos de ciclo de vida representan **distintas formas de organizar el avance del desarrollo**. Se distinguen tres familias principales: **secuencial, iterativa y recursiva**.

No existe un modelo universalmente mejor. La elección depende del contexto: estabilidad de los requerimientos, incertidumbre, riesgos, necesidad de obtener valor temprano, posibilidad de recibir retroalimentación, restricciones de tiempo y recursos, conocimiento técnico y características del producto.

1- SECUENCIAL Y CASCADA

En el enfoque **secuencial**, las actividades se realizan principalmente una después de otra. Este enfoque permite **planificar anticipadamente, establecer hitos y distribuir responsabilidades con claridad**. Estas características hacen que el enfoque sea útil cuando los requerimientos y la tecnología son conocidos y estables.

En la cascada pura, por ejemplo, se avanza desde requerimientos hacia diseño, implementación y validación, cerrando principalmente una etapa antes de avanzar a la siguiente.

Su **principal dificultad aparece cuando existe incertidumbre**: muchas decisiones deben tomarse al comienzo y los errores o cambios descubiertos tarde pueden obligar a rehacer trabajo de varias etapas. Además, **la evidencia del funcionamiento real del producto suele aparecer más tarde** que en enfoques incrementales.

VARIANTES DE CASCADA

Existen variantes que intentan disminuir la rigidez sin abandonar la organización predominantemente secuencial, por lo que **no eliminan el problema fundamental de tomar una parte importante de las decisiones anticipadamente**:

- **Con retroalimentación**: Permite regresar a una fase anterior cuando se detecta un problema. Facilita corregir, aunque el cambio sigue siendo más costoso cuanto más tarde se descubre.
- **Con fases solapadas**: Una etapa puede comenzar antes de finalizar completamente la anterior. Reduce esperas, pero aumenta la necesidad de coordinación y el riesgo de retrabajo.
- **Con subproyectos**: Luego de definir una base común, distintas partes pueden desarrollarse en paralelo. Puede reducir tiempos, pero exige controlar dependencias e integración.

2- ITERATIVO E INCREMENTAL

Iterar significa volver sobre la solución para revisarla y mejorarla. **Incrementar** significa agregar nuevas capacidades funcionales a lo ya construido.

Ambas ideas pueden combinarse: en cada ciclo se desarrolla una parte del producto, se observa el resultado, se obtiene retroalimentación y lo aprendido influye sobre los ciclos siguientes.

Esto permite **entregar valor antes, adaptarse a cambios y reducir la cantidad de decisiones que deben quedar cerradas desde el inicio**. También facilita que usuarios y equipo descubran problemas a partir de software funcionando.

Como contrapartida, requiere gestionar adecuadamente las dependencias entre incrementos, conservar una arquitectura coherente y evitar que los cambios sucesivos degraden la calidad del producto.

ENTREGA POR ETAPAS Y ENTREGA EVOLUTIVA

Ambas permiten entregar el producto progresivamente, pero se diferencian en cuánto aprendizaje modifica lo que sigue:

- En la **entrega por etapas**, las partes que se entregarán están principalmente planificadas de antemano. Permite obtener producto utilizable antes de finalizar todo el desarrollo, pero tiene menor capacidad para absorber cambios profundos.
- En la **entrega evolutiva**, cada versión aporta retroalimentación que influye sobre cómo continuará el producto. Es útil cuando es necesario aprender a partir del uso, aunque exige participación del usuario y control del alcance.

Por lo tanto, **entregar en partes no implica necesariamente trabajar de manera evolutiva**: la diferencia está en cuánto puede modificar el aprendizaje posterior el desarrollo previsto.

PROTOTIPADO

Un prototipo se construye principalmente para **aprender antes de comprometer una inversión mayor**. Puede utilizarse para aclarar necesidades, explorar una interacción o reducir incertidumbre técnica. Su valor está en hacer observable una idea para poder obtener evidencia y retroalimentación.

Existen dos formas principales:

- **Prototipo desechable**: Se construye para aprender y luego se descarta.
- **Prototipación evolutiva**: La versión inicial se mejora progresivamente hasta convertirse en el producto.

El **riesgo principal es confundir un prototipo con software listo para producción**. Una solución construida rápidamente para aprender puede no poseer todavía la arquitectura, calidad, seguridad o mantenibilidad necesarias para convertirse directamente en producto.

3- MODELO RECURSIVO Y ESPIRAL

Dentro de la familia recursiva, el principal modelo estudiado es la espiral. **Su característica central es que el avance está dirigido por los riesgos**: cada vuelta busca identificar y reducir primero aquello que puede comprometer el proyecto.

En cada ciclo se:

1. Establecen objetivos, alternativas y restricciones;
2. Identifican y evalúan los riesgos más relevantes;
3. Se produce evidencia para reducirlos y se desarrolla o valida lo necesario;
4. Se revisa lo aprendido y se planifica la siguiente vuelta.

Por eso, **una vuelta de la espiral no tiene que producir necesariamente una versión del producto**: puede producir un prototipo, una prueba técnica o conocimiento suficiente para tomar una decisión.

Su principal ventaja es **enfrentar temprano los riesgos importantes antes de comprometer grandes recursos**. Resulta especialmente útil cuando existen riesgos técnicos, de viabilidad o criticidad elevados. Como contrapartida, identificar y gestionar riesgos **requiere experiencia y puede introducir un costo innecesario en proyectos pequeños y bien conocidos**.

4- OTRAS ALTERNATIVAS DE DESARROLLO

Además de las familias principales, existen estrategias que pueden resultar adecuadas ante restricciones específicas.

1- Code and Fix: Se comienza a programar y se corrigen los problemas a medida que aparecen, sin suficiente planificación o diseño previo. Puede servir para desarrollos muy pequeños y descartables, pero al crecer genera retrabajo, baja predictibilidad y pérdida de coherencia. No debe confundirse con agilidad, que sí incorpora planificación, disciplina y retroalimentación sistemática.

2- Diseño para cronograma. Se utiliza cuando la **fecha es fija pero el alcance puede negociarse**. Se priorizan las funcionalidades más importantes para asegurar una versión útil dentro del tiempo disponible.

3- Diseño para herramientas. Se aprovechan herramientas, bibliotecas o componentes existentes para acelerar el desarrollo. A cambio, parte de las funcionalidades y decisiones técnicas quedan condicionadas por esas herramientas y puede aparecer dependencia de terceros.

4- Software comercial o reutilización. Antes de construir una solución desde cero se evalúa si existe una alternativa que pueda comprarse, configurarse o reutilizarse. Deben considerarse su adecuación a la necesidad, integración, adaptación, costos y dependencia del proveedor.

SELECCIÓN Y JUSTIFICACIÓN DEL CICLO DE VIDA

La elección del ciclo de vida depende principalmente de **qué incertidumbre o restricción domina el proyecto**.

Situación predominante	Enfoque a considerar	Fundamento
Requerimientos claros y estables	Secuencial	Permite planificar anticipadamente con menor riesgo de retrabajo.
Cambios frecuentes o necesidad de valor temprano	Iterativo incremental	Permite entregar, obtener retroalimentación y adaptar lo siguiente.
Incertidumbre sobre necesidades o interacción	Prototipado / evolución	Permite aprender antes de comprometer grandes esfuerzos.
Riesgos técnicos o de viabilidad importantes	Espiral	Organiza el desarrollo alrededor de la reducción temprana del riesgo.
Fecha fija y alcance negociable	Diseño para cronograma	Prioriza una versión útil dentro del tiempo disponible.
Existe una solución o herramienta adecuada	Compra, reutilización o diseño para herramientas	Evita desarrollar desde cero capacidades que ya están disponibles.

La **elección del ciclo de vida cambia la forma de gestionar el proyecto** ya que se determina cuándo se detallan requerimientos, cuándo se comprometen recursos, qué evidencia se utiliza para observar el avance, cuándo participa el usuario y cómo se enfrentan los riesgos.

Por eso, ante una situación concreta no alcanza con nombrar un modelo. Una justificación completa debe relacionar:

Contexto del proyecto → Característica del modelo → Beneficio esperado → Costo o riesgo que se acepta.

Por ejemplo, si los requerimientos cambian y el usuario puede validar frecuentemente, un enfoque iterativo incremental permite aprender y ajustar a partir de entregas parciales, aunque exige gestionar correctamente la arquitectura y las dependencias entre incrementos.

ADMINISTRACIÓN DE PROYECTOS DE SOFTWARE

Administrar un proyecto consiste en **organizar y coordinar el trabajo para alcanzar sus objetivos dentro de las restricciones existentes**. Implica aplicar conocimientos, habilidades, herramientas y técnicas; identificar necesidades; establecer objetivos claros y alcanzables; y adaptar planes y decisiones a los distintos involucrados.

La gestión no consiste únicamente en realizar una planificación inicial. Incluye dos actividades permanentes:

- **Planificación:** decidir cómo se pretende alcanzar los objetivos.
- **Monitoreo y control:** Observar lo que realmente ocurre y actuar cuando aparecen diferencias respecto de lo esperado.

Una buena gestión permite crear **mejores condiciones para alcanzar el éxito, pero no lo garantiza**.

Son **factores favorables** los requerimientos claros, participación de usuarios, apoyo de la dirección, expectativas realistas, planificación adecuada, objetivos e hitos claros, comunicación y personas competentes.

Entre los **factores desfavorables** se encuentran definir mal el problema, trabajar con información insuficiente, asignar inadecuadamente los recursos, no realizar seguimiento y estimar sin utilizar evidencia o datos históricos.

ALCANCE Y TRIPLE RESTRICCIÓN

Alcance del producto: Describe las características y funciones que debe poseer la solución. Su cumplimiento se evalúa respecto de los requerimientos.

Alcance del proyecto: Comprende todo el trabajo, y sólo el trabajo, necesario para obtener ese resultado. Su cumplimiento se evalúa respecto del plan de proyecto.

Por eso, modificar el producto normalmente afecta al proyecto: agregar una funcionalidad puede requerir nuevo análisis, desarrollo, pruebas, documentación, integración o despliegue.

La **triple restricción** relaciona tres variables que **se condicionan mutuamente y su equilibrio repercute en la calidad**. Si una cambia, debe analizarse el impacto sobre las restantes:

- **Alcance:** qué se pretende obtener;
- **Tiempo:** cuándo debe estar disponible;
- **Costo:** qué recursos económicos pueden utilizarse.

La **tarea de gestión consiste en hacer visibles y balancear esas restricciones**, no en conservar artificialmente todas las variables a costa de deteriorar la calidad. Por ejemplo, aumentar alcance manteniendo la misma fecha y presupuesto no hace desaparecer el trabajo adicional: puede exigir renegociar prioridades, recursos o plazos.

PERSONAS, EQUIPO Y COORDINACIÓN

El desarrollo de software depende fuertemente de las personas. Un **equipo de proyecto** reúne conocimientos y habilidades diferentes alrededor de objetivos compartidos y responsabilidad conjunta. La coordinación permite integrar esos aportes en un resultado coherente.

El **líder de proyecto** facilita esa coordinación y conecta al equipo con clientes, dirección, proveedores y demás interesados o **stakeholders**, es decir, personas u organizaciones que pueden afectar al proyecto o verse afectadas por él.

A medida que aumenta el equipo también aumenta el esfuerzo necesario para coordinarlo. Con n personas existen $n \times (n - 1) / 2$ pares posibles de comunicación. Esto no significa que todos se comuniquen permanentemente entre sí, sino que muestra por qué **agregar personas también introduce nuevas necesidades de comunicación y organización.**

Por esa razón, **esfuerzo y duración no son intercambiables.** El mes-persona es una unidad de esfuerzo: representa aproximadamente el trabajo de una persona durante un mes. No implica que cualquier tarea pueda realizarse en la mitad del tiempo duplicando la cantidad de personas, porque existen actividades secuenciales, dependencias, aprendizaje y coordinación.

Esto explica la **ley de Brooks**: incorporar personas a un proyecto de software ya retrasado puede retrasarlo todavía más cuando quienes ingresan deben capacitarse y coordinarse con quienes ya estaban trabajando. No significa que aumentar capacidad sea siempre perjudicial; depende del momento y de cuánto pueda dividirse realmente el trabajo.

COHERENCIA Y COMPLEJIDAD DE LA SOLUCIÓN

Gestionar el proyecto también implica evitar que el trabajo de muchas personas produzca un producto formado por decisiones incompatibles.

La **integridad conceptual** significa que las distintas partes del sistema deben responder a una visión coherente, es decir, la distribución del trabajo no debería fragmentar las decisiones centrales de diseño. Esto no implica que no se puedan utilizar distintas estructuras organizativas, siempre y cuando se mantengan criterios compartidos y responsabilidades claras sobre las decisiones fundamentales.

El **efecto del segundo sistema** es la tendencia a incorporar en una nueva solución todas las ideas, características y mejoras que quedaron fuera de la anterior. Esto puede producir un sistema innecesariamente complejo. Por eso, cada nueva capacidad debe justificarse según el propósito del producto, su valor y las restricciones existentes.

La complejidad tampoco se resuelve únicamente escribiendo más código. Una adecuada **representación y organización de los datos puede simplificar significativamente la solución**, evitando algoritmos complicados creados sólo para compensar una estructura deficiente.

PLANIFICACIÓN Y PLAN DE PROYECTO

El **plan de proyecto** documenta las decisiones principales sobre: **qué se hará → cuándo → cómo → quién será responsable.** Funciona como referencia para coordinar el trabajo y posteriormente comparar lo planificado con lo que realmente ocurre.

La planificación incluye principalmente:

- Definición del alcance;
- Elección del proceso y ciclo de vida;
- Estimaciones;
- Gestión de riesgos;
- Asignación de recursos;
- Programación de actividades;
- Definición de controles y métricas.

Estas decisiones están relacionadas y se realizan con la información disponible en cada momento.

Por eso, **planificar es más importante que tratar al plan inicial como algo inmutable.** Cuando aparece nueva información, el plan puede revisarse conscientemente manteniendo registro de qué cambió y por qué.

ESTIMACIONES

Una **estimación** es una predicción sobre una magnitud futura. En proyectos de software pueden estimarse, entre otras cosas:

- tamaño del producto;
- esfuerzo;
- duración o calendario;
- costo;
- recursos críticos.

Una estimación **no es una planificación ni un compromiso**. Las estimaciones proporcionan información para construir planes, pero el plan incorpora además objetivos, restricciones y decisiones de gestión. Si el plan se aparta significativamente de una estimación realista, esa diferencia representa un riesgo que debe hacerse visible y gestionarse.

Como existe incertidumbre, una estimación no debe interpretarse como un valor exacto. Cuanto menor es el conocimiento disponible, mayor suele ser el margen de error. Las estimaciones deben revisarse cuando aparece nueva información.

Entre las causas frecuentes de error se encuentran trabajar con información insuficiente, omitir actividades necesarias o ignorar datos históricos. Por eso la calidad de una estimación depende tanto del método utilizado como de la información disponible.

GESTIÓN DE RIESGOS

Un **riesgo** es un evento o condición incierta que, si ocurre, puede afectar los objetivos del proyecto. Se diferencia de un **problema**, que ya ocurrió y debe ser tratado.

Gestionar riesgos implica:

1. **planificar su gestión**
2. **identificarlos**
3. **analizar probabilidad e impacto**
4. **definir respuestas**
5. **realizar seguimiento y control.**

La gestión de riesgos se realiza durante todo el proyecto porque tanto las condiciones como la información disponible cambian.

Por ejemplo, pueden existir riesgos asociados a requerimientos inestables, dependencia de personas clave, tecnologías desconocidas o integraciones externas. Las respuestas dependen del riesgo: un prototipo puede reducir incertidumbre técnica, una validación temprana puede reducir incertidumbre sobre requerimientos y la transferencia de conocimiento puede reducir la dependencia de una persona.

La finalidad no es eliminar toda incertidumbre, sino **hacer visibles los riesgos relevantes y reducir o controlar su exposición antes de que se conviertan en problemas**.

MÉTRICAS, MONITOREO Y CONTROL

Una **métrica** aporta evidencia objetiva sobre algún aspecto del desarrollo. Se distinguen:

- **Métricas de producto:** Describen características del resultado.
- **Métricas de proyecto:** Permiten observar cómo se está ejecutando un proyecto concreto.
- **Métricas de proceso:** Utilizan información de distintos proyectos para analizar y mejorar la forma de trabajar de la organización.

Entre las métricas básicas de un proyecto aparecen tamaño, esfuerzo, tiempo calendario y defectos.

Medir no significa acumular números. Una métrica es útil si **aporta información nueva, tiene utilidad práctica y ayuda a responder una pregunta necesaria para decidir.**

El **monitoreo** observa el estado del proyecto y compara lo real con lo planificado. Detectar una desviación no es suficiente: el **control** consiste en decidir e implementar una respuesta, por ejemplo, modificar prioridades, reorganizar trabajo, tratar un riesgo, revisar recursos o actualizar pronósticos.

El objetivo no es modificar retrospectivamente el plan para que parezca que nunca hubo una desviación, sino **comprender qué cambió, por qué cambió y qué decisión debe tomarse a partir de esa evidencia.**

U2: Producto, requerimientos, estimaciones y Scrum

AGILIDAD Y GESTIÓN DE PRODUCTO

La **agilidad es una filosofía de trabajo basada en valores y principios** que orientan el desarrollo de productos. No es por sí misma una metodología ni significa simplemente trabajar en períodos cortos.

Busca **responder al cambio, colaborar con quienes necesitan el producto y obtener evidencia frecuente sobre si lo construido realmente aporta valor.**

Por eso, un **enfoque ágil** combina planificación continua, equipos con capacidad de decisión, entregas frecuentes y priorizadas y prácticas técnicas disciplinadas. **Adaptarse no significa improvisar** y reducir documentación innecesaria no significa eliminar toda documentación.

VALORES DEL MANIFIESTO ÁGIL

El Manifiesto Ágil establece cuatro preferencias:

1. **Individuos e interacciones** sobre procesos y herramientas.
2. **Software funcionando** sobre documentación exhaustiva.
3. **Colaboración con el cliente** sobre negociación contractual.
4. **Respuesta al cambio** sobre seguimiento de un plan.

Los elementos de la derecha también tienen valor. La idea es que, cuando existe una tensión, **se prioriza aquello que facilita aprender, colaborar y entregar valor.**

Por ejemplo, un plan sigue siendo necesario, pero debe poder revisarse cuando aparece nueva información; la documentación sigue siendo útil cuando ayuda a comprender y mantener el producto, pero no debe convertirse en un fin en sí misma.

PRINCIPIOS ÁGILES

Los doce principios del Manifiesto pueden entenderse mediante algunas ideas centrales:

- **Entregar valor temprano y frecuentemente**, utilizando software funcionando como principal evidencia de progreso.
- **Aceptar el cambio**, incluso durante el desarrollo, cuando mejora el resultado.
- **Mantener colaboración frecuente** entre negocio y quienes construyen la solución.
- Trabajar con **personas motivadas y equipos autoorganizados**, con responsabilidad sobre el resultado.
- Mantener un **ritmo sostenible, excelencia técnica y buen diseño** para poder seguir adaptando el producto.
- Aplicar **simplicidad**, evitando trabajo que no aporte valor.
- **Reflexionar periódicamente y adaptar la forma de trabajar.**

Estas ideas están relacionadas: entregar frecuentemente permite obtener evidencia; colaborar permite interpretarla; y mantener calidad técnica permite modificar la solución sin que cada cambio sea cada vez más costoso.

DE PRODUCIR FUNCIONES A GENERAR VALOR

La gestión de producto no se concentra únicamente en **qué funcionalidades construir**, sino en determinar:

qué problema resolver → para quién → mediante qué propuesta de valor → y cómo comprobar que la solución funciona.

Una funcionalidad implementada es una **salida (output)**. El cambio que esa funcionalidad produce en el comportamiento de los usuarios o en el negocio es un **resultado (outcome)**.

Por eso: **más funcionalidades entregadas ≠ necesariamente más valor generado.**

Un equipo puede implementar correctamente muchas características que luego nadie utiliza. El objetivo no es maximizar la cantidad de software producido, sino **construir el producto adecuado y comprobar que realmente satisface una necesidad.**

Esta visión también explica la diferencia entre proyecto y producto vista anteriormente: un proyecto puede finalizar, mientras que el producto continúa operando, generando evidencia y evolucionando.

HIPÓTESIS Y APRENDIZAJE VALIDADO

Cuando todavía existe incertidumbre sobre el producto, muchas decisiones deben tratarse como **hipótesis que necesitan evidencia.**

Una **hipótesis de valor** expresa por qué determinada solución debería resultar útil para un usuario y qué evidencia permitiría comprobarlo.

Una **hipótesis de crecimiento** se pregunta cómo nuevos usuarios descubrirán o adoptarán el producto y cómo puede sostenerse o ampliarse su uso.

Son problemas diferentes: demostrar que algunos usuarios valoran una solución no demuestra automáticamente que el producto pueda crecer.

Para reducir incertidumbre se utiliza el ciclo: **Construir → Medir → Aprender**. Se construye un producto o experimento suficientemente pequeño, se observa qué ocurre realmente y esa evidencia se utiliza para confirmar la hipótesis o modificarla.

El objetivo es lograr **aprendizaje validado**: reemplazar suposiciones por información obtenida del comportamiento real de los usuarios.

Retrasar indefinidamente el contacto con usuarios puede parecer más seguro porque evita resultados negativos, pero también impide aprender y puede hacer que se descubra demasiado tarde que se estaba construyendo algo incorrecto.

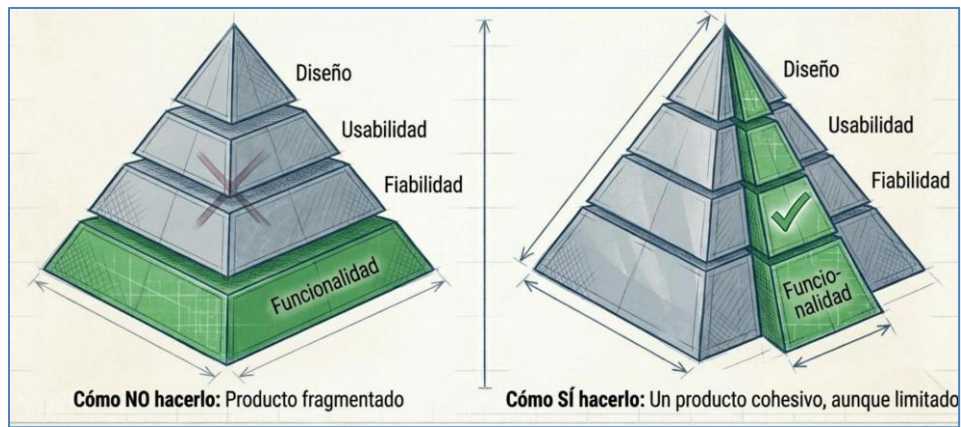
PRODUCTO MÍNIMO VIABLE (MVP)

Un MVP es el mínimo producto, versión o experimento necesario para **poner a prueba una hipótesis relevante y obtener evidencia de usuarios o potenciales usuarios utilizando un esfuerzo proporcionado.**

Lo importante de “mínimo” no es simplemente construir pocas funcionalidades, sino **construir sólo lo necesario para obtener el aprendizaje buscado.**

Tampoco significa entregar un producto de mala calidad. Si el experimento requiere que el usuario utilice realmente la solución, ésta necesita suficiente funcionalidad, fiabilidad, usabilidad y diseño para que la experiencia permita evaluar correctamente la hipótesis.

El MVP debe concentrarse en la **Propuesta de Valor Única (UVP)**: aquello que representa la razón principal por la cual el usuario debería elegir la solución.



La imagen muestra una idea importante: **un MVP debe ser un recorte vertical y coherente de la experiencia**, aunque limitado. Construir solamente una capa —por ejemplo, toda la funcionalidad, pero sin suficiente fiabilidad o usabilidad— puede impedir que el usuario experimente realmente la propuesta de valor.

La forma concreta del MVP depende de **qué hipótesis se quiere validar**. Puede utilizarse, por ejemplo:

- una demostración o video;
- un prototipo;
- una prueba de humo;
- una versión funcional limitada.

Cada experimento aporta evidencia distinta. Una demostración puede probar interés, pero no necesariamente demuestra que los usuarios utilizarán el producto sostenidamente o que la solución técnica pueda escalar.

Por eso, lo importante es comprender: **Hipótesis → Experimento adecuado → Evidencia → Decisión**.

PROTOTIPO, PRUEBA DE HUMO Y PIVOTE

Un **prototipo** representa parte de la solución o experiencia para explorar una idea y obtener aprendizaje antes de desarrollar el producto completo.

Una **prueba de humo**, dentro de gestión de producto, presenta una propuesta a potenciales usuarios para observar su interés o intención **antes de disponer completamente de la solución**. No debe confundirse con el significado técnico de *smoke test* utilizado en testing.

Un **pivote** es un cambio significativo en la hipótesis o dirección del producto realizado a partir de evidencia.

No significa simplemente aceptar cualquier pedido de un usuario. Si cada persona solicita algo diferente, puede indicar que todavía no existe una necesidad común bien identificada. En cambio, si la evidencia muestra repetidamente el mismo problema, existe una señal más fuerte para modificar la hipótesis o la solución.

PRIORIZACIÓN

Como los recursos son limitados, no todo debe incorporarse al producto inmediatamente.

La priorización debe considerar **qué capacidades son necesarias** para probar o entregar la propuesta de valor y cuáles pueden **postergarse, delegarse o eliminarse**. Se propone distinguir entre:

1. **Funcionalidades críticas** que deben incluirse;
2. **Funcionalidades importantes pero que pueden esperar**;
3. Capacidades que pueden resolverse **mediante servicios o integraciones externas**;
4. **Trabajo que no aporta suficiente valor** y debería eliminarse.

La herramienta concreta utilizada para priorizar es secundaria. Lo importante es **evitar construir por defecto todo lo solicitado y justificar cada elemento respecto del objetivo del producto**.

PRODUCTOS Y CARACTERÍSTICAS MÍNIMAS

La cátedra utiliza distintos conceptos “mínimos”. Todos buscan limitar el alcance, pero **responden a preguntas diferentes**.

Concepto	Pregunta principal
MVP Minimum Viable Product	¿Cuál es el mínimo experimento/producto necesario para validar una hipótesis y aprender?
MVF Minimum Viable Feature	¿Cuál es la mínima función que permite explorar una nueva hipótesis dentro del producto?
MMF Minimum Marketable Feature	¿Cuál es la mínima funcionalidad con suficiente valor comercial para poder liberarse?
MMP Minimum Marketable Product	¿Cuál es el producto mínimo suficientemente completo para realizar el lanzamiento comercial?
MMR Minimum Marketable Release	¿Cuál es el conjunto mínimo de capacidades que justifica una nueva liberación al mercado?
MLP Minimum Lovable Product	¿Cuál es la experiencia mínima que, además de resolver la necesidad, puede generar preferencia y vínculo con el usuario?

La diferencia fundamental es el **objetivo del recorte**:

MVP/MVF → Aprender y validar

MMF/MMP/MMR → Entregar valor comercial

MLP → Generar una experiencia que favorezca preferencia y continuidad de uso

Estas categorías tampoco deben confundirse con las **User Stories**. Una User Story representa una unidad de necesidad/trabajo manejable por el equipo; puede contribuir a un MVP, MMF o MMP, pero no es automáticamente equivalente a ellos.

ROADMAP Y EVOLUCIÓN DEL PRODUCTO

Un **roadmap** comunica la dirección prevista de evolución del producto. Puede mostrar el recorrido desde experimentación y aprendizaje hasta lanzamiento y posteriores releases.

No constituye una promesa inmutable ni supone que después del lanzamiento desaparezca la incertidumbre. El producto continúa generando evidencia y pueden aparecer nuevas hipótesis que requieran nuevamente experimentación.

Por eso, el aprendizaje **no es únicamente una fase previa al lanzamiento**, sino una actividad que acompaña toda la vida del producto.

MINIMUM LOVABLE PRODUCT (MLP)

El **MLP** incorpora una preocupación adicional: no sólo que el producto funcione y satisfaga una necesidad, sino que la experiencia sea suficientemente positiva como para favorecer **preferencia, confianza y continuidad de uso**.

No significa simplemente agregar estética o funciones decorativas. Una experiencia “lovable” debe resolver el problema con baja fricción y generar una percepción positiva del producto.

Existen distintas dimensiones que pueden contribuir a esa experiencia. No las interpretaría como una receta ni como una secuencia de implementación: sirven para pensar que el valor percibido incluye elementos funcionales y también la experiencia completa del usuario.

La diferencia conceptual más importante es:

MVP → ¿qué necesito para aprender?

MMP → ¿qué necesito para comercializar?

MLP → ¿qué experiencia mínima puede hacer que el usuario valore y prefiera el producto?

No son tres grados automáticos de calidad ni etapas que todos los productos deban atravesar obligatoriamente.

REQUERIMIENTOS ÁGILES Y USER STORIES

En un enfoque ágil, los requerimientos se **elaboran progresivamente**. Se mantiene una visión general del producto, pero el detalle se incorpora cuando el trabajo está próximo a realizarse.

Esto permite utilizar nueva información y evita invertir demasiado esfuerzo en especificaciones que pueden cambiar o incluso descartarse.

La crítica al **BRUF —Big Requirements Up Front—** no significa desarrollar sin comprender qué se necesita. Significa evitar intentar definir exhaustivamente todo el producto desde el comienzo, como si las necesidades fueran completamente conocidas y estables.

El objetivo es alcanzar **el nivel de detalle suficiente para tomar la siguiente decisión y realizar el trabajo próximo**, refinando posteriormente cuando sea necesario.

TIPOS DE REQUERIMIENTOS

Los distintos requerimientos expresan necesidades en diferentes niveles:

- **De negocio:** Describen un resultado que la organización busca conseguir.
- **De usuario:** Describen algo que una persona necesita lograr.
- **Funcionales:** Indican comportamientos o servicios que debe ofrecer el sistema.
- **No funcionales:** Establecen cualidades o restricciones sobre el comportamiento.
- **De implementación:** Condicionan decisiones técnicas de la solución.

Por ejemplo:

reducir el tiempo de atención → negocio

consultar el estado de una cuenta → usuario

generar un reporte → funcional

responder antes de cierto tiempo → no funcional

utilizar determinada plataforma → implementación

Estos niveles están relacionados, pero **no son distintas formas de escribir exactamente el mismo requerimiento**. Se utiliza esta clasificación para distinguir el problema que se quiere resolver de las decisiones concretas de la solución.

USER STORY

Una User Story **expresa brevemente una necesidad desde la perspectiva de quien recibirá valor**. No pretende especificar exhaustivamente cómo funcionará el sistema. Funciona como **recordatorio de una necesidad y punto de partida para una conversación** que permitirá comprenderla mejor antes de implementarla.

El formato utilizado por la cátedra es: **Como [rol], quiero/puedo [actividad], para [beneficio o valor]**.

- **Rol:** quién necesita la capacidad.
- **Actividad:** qué necesita hacer.
- **Beneficio:** para qué o qué valor obtiene.

El valor de la historia no está en completar mecánicamente esos tres campos. Una buena historia permite comprender **quién necesita qué y por qué**.

Por eso, no todo trabajo debe forzarse al formato de User Story. Una tarea técnica puede ser necesaria sin representar directamente una necesidad del usuario, y un Product Backlog puede contener otros tipos de trabajo.

LAS 3 C: CARD, CONVERSATION Y CONFIRMATION

Una User Story no es solamente la frase escrita en una tarjeta. Se apoya en tres componentes:

- **Card:** Resume la intención de manera breve. No reemplaza una especificación completa.
- **Conversation:** Permite aclarar reglas, restricciones, alternativas, ejemplos y dudas. Gran parte del conocimiento necesario para implementar la historia surge de este intercambio.
- **Confirmation:** Define cómo comprobar que se logró lo acordado mediante criterios y pruebas de aceptación.

Por lo tanto:

tarjeta breve + conversación compartida + forma de comprobar el resultado = comprensión suficiente de la historia.

La brevedad de una User Story no implica falta de análisis; implica que **el conocimiento no se concentra exclusivamente en el texto de la tarjeta.**

ROLES Y USUARIOS

Las historias deben escribirse desde la perspectiva de un **rol relevante**, evitando utilizar siempre un “usuario” genérico.

Diferentes usuarios pueden tener objetivos, conocimientos y contextos distintos, y esas diferencias pueden modificar lo que realmente necesitan.

Cuando no se dispone de acceso directo al usuario pueden utilizarse representantes o **proxies**, como expertos del dominio, soporte, ventas o clientes. Estos aportan información útil, pero también pueden introducir sesgos.

Por eso, **cliente, comprador, usuario, representante y Product Owner no son necesariamente la misma persona.**

CRITERIOS Y PRUEBAS DE ACEPTACIÓN

Los **criterios de aceptación** establecen las condiciones que debe cumplir una historia para satisfacer la necesidad acordada. Deben evitar imponer una implementación técnica cuando no sea necesario. Sirven para:

- Delimitar el alcance de la historia;
- Generar comprensión compartida entre Product Owner, desarrolladores y testers;
- Derivar pruebas;
- Evitar continuar agregando funcionalidad innecesariamente.

Una **prueba de aceptación** convierte esas condiciones en un caso concreto con datos y un resultado esperado.

Por lo tanto:

criterio de aceptación = qué condición debe cumplirse

prueba de aceptación = cómo comprobamos concretamente esa condición

Es importante no confundir el resultado esperado de la operación con el resultado de la prueba: **una prueba que introduce un dato inválido pasa si el sistema lo rechaza correctamente.** La presentación utiliza justamente casos válidos e inválidos para comprobar una misma historia.

INVEST: CALIDAD DE UNA USER STORY

El modelo **INVEST** permite evaluar si una historia está formulada de una manera útil para el equipo. No garantiza automáticamente que una historia sea correcta, pero ayuda a detectar problemas que dificultarían su **priorización, estimación, implementación o validación.**

Independent

Debe tener la menor cantidad posible de dependencias rígidas respecto de otras historias para poder ordenarla e implementarla con flexibilidad. Si existe una dependencia real debe hacerse visible, no ignorarse.

Negotiable

La historia expresa principalmente **qué se necesita**, dejando espacio para discutir **cómo resolverlo**. No es un contrato técnico cerrado.

Valuable

Debe ser posible explicar qué valor aporta al usuario o cliente. Una tarea técnica no adquiere valor de usuario simplemente por escribirla con formato de historia.

Estimatable

El equipo debe comprenderla suficientemente como para estimar su tamaño relativo. Si no puede hacerlo, puede existir incertidumbre sobre el problema, la solución o el alcance.

Small

Debe poder completarse dentro de una iteración. Si es demasiado grande debe dividirse en capacidades más pequeñas y útiles.

Testable

Debe existir una forma observable de demostrar que fue implementada correctamente. Expresiones ambiguas como “rápido” o “fácil de usar” necesitan precisarse.

DIVISIÓN DE HISTORIAS Y CORTE VERTICAL

Cuando una historia es demasiado grande, debe dividirse sin perder de vista el valor para el usuario.

Un **corte vertical** atraviesa las diferentes capas técnicas necesarias para entregar una capacidad completa, por ejemplo: interfaz + lógica de negocio + datos. Distintas historias pueden atravesar las mismas capas técnicas, pero ofrecer capacidades diferentes al usuario.

Un **corte horizontal**, en cambio, implementa solamente una capa técnica y todavía no permite obtener el comportamiento completo. Por sí solo, esto no constituye una buena división de una historia por valor.

NIVELES DE ABSTRACCIÓN

El trabajo también puede expresarse con diferentes niveles:

- Un **Theme** agrupa necesidades relacionadas.
- Una **Epic** es una necesidad demasiado grande para implementarse directamente y debe dividirse.
- Una **User Story** representa una porción manejable de necesidad o valor.
- Una **Task** describe trabajo técnico necesario para implementar una historia.

Una tarea técnica, por lo tanto, **no debe confundirse con una necesidad del usuario**.

Los elementos más lejanos pueden permanecer más grandes y poco detallados; los próximos a desarrollarse necesitan mayor claridad y un tamaño manejable. Esto es coherente con la elaboración progresiva de requerimientos.

DEFINITION OF READY, CRITERIOS DE ACEPTACIÓN Y DEFINITION OF DONE

Estos conceptos responden a preguntas diferentes y no deben confundirse.

Criterios de aceptación → ¿qué comportamiento o condición debe cumplir esta historia?

Son específicos de cada historia.

Definition of Ready — DoR → ¿tenemos suficiente comprensión para comenzar a trabajar sobre este ítem?

Puede considerar que el propósito esté entendido, que el tamaño sea razonable, que los criterios de aceptación hayan sido discutidos y que se conozcan dependencias relevantes.

“Ready” no significa que ya se conozca absolutamente todo ni que no pueda aparecer aprendizaje posteriormente. En el enfoque 2026 se entiende como una calidad que surge del refinamiento, no como un compromiso formal de Scrum.

Definition of Done — DoD → ¿el trabajo terminado cumple el estándar de calidad común necesario para formar parte del producto?

La DoD se aplica al incremento y puede incluir integración, pruebas, documentación, seguridad u otras condiciones de calidad. Por eso, una historia podría satisfacer sus criterios de aceptación y aun así **no estar terminada** si incumple la DoD.

La diferencia clave es:

Criterios de Aceptación → específicos de la historia

DoR → preparación antes de comenzar

DoD → calidad necesaria para considerar terminado el resultado

SPIKES

Un **spike** es una investigación acotada cuyo objetivo es **reducir una incertidumbre importante antes de comprometerse con una solución o estimación**. Puede ser:

- **Técnico:** investigar una tecnología, rendimiento, arquitectura, integración o decisión de hacer/comprar.
- **Funcional:** investigar cómo debería comportarse la solución o cómo interactuarán los usuarios con ella.

Un spike debe tener **un objetivo concreto, un límite y un resultado que permita tomar una decisión**. Puede producir una conclusión, comparación o prototipo; no necesita entregar una funcionalidad al usuario.

No debe utilizarse para investigar indefinidamente toda incertidumbre. La propia cátedra lo presenta como **una excepción para incógnitas importantes**, ya que cierto grado de incertidumbre forma parte normal del desarrollo.

Normalmente se realiza antes de la historia que depende de su resultado. Si la investigación es pequeña y puede resolverse rápidamente, podría realizarse dentro de la misma iteración.

CÓMO RECONOCER UNA HISTORIA MAL FORMULADA

Una historia merece revisión cuando:

- combina varios objetivos independientes mediante “y”;
- se concentra únicamente en detalles de implementación;
- utiliza un rol genérico sin necesidad;
- no permite identificar qué valor aporta;
- tiene un tamaño demasiado grande;
- depende rígidamente de muchas otras historias;
- utiliza criterios vagos o imposibles de verificar;
- intenta contener en la tarjeta toda la especificación necesaria.

La cátedra resume varias de estas ideas recomendando **un paso a la vez, palabras claras en los criterios, perspectiva del usuario, conversación y no forzar todo a ser una User Story**.

La pregunta práctica al revisar una historia debería ser:

¿entiendo quién necesita qué y por qué, puedo conversar lo necesario, sé cómo comprobarlo y el trabajo tiene un tamaño que permite entregar una capacidad útil?

ESTIMACIÓN Y PLANIFICACIÓN ÁGIL

Una **estimación** es una predicción razonada sobre una magnitud futura y siempre contiene incertidumbre. No debe confundirse con:

- Un **objetivo**, que expresa un resultado deseado;
- Un **compromiso**, que es un acuerdo bajo determinadas condiciones;
- Un **plan**, que organiza las acciones para alcanzar un objetivo o compromiso.

Por eso, si el negocio desea terminar en una fecha, pero la estimación indica otra duración, **no se resuelve el problema modificando arbitrariamente la estimación**. Deben revisarse alcance, recursos, condiciones, prioridades o el riesgo que se está dispuesto a asumir.

La **estimación puede revisarse cuando aparece nueva información relevante sobre el trabajo**. El compromiso, en cambio, puede negociarse.

Además, estimar no sirve únicamente para obtener un número: el propio proceso permite discutir supuestos, detectar trabajo omitido, descubrir incertidumbre y evaluar si lo que se pretende realizar es factible.

INCERTIDUMBRE

Al comienzo de un desarrollo existe mayor incertidumbre porque todavía se conoce menos sobre el producto, la solución y los riesgos. A medida que se aclaran requerimientos, se validan hipótesis, se resuelven dudas técnicas y se obtiene experiencia real, las estimaciones pueden volverse más confiables.

Por eso, **la incertidumbre no disminuye simplemente porque pase el tiempo**, sino porque se obtiene conocimiento.

Entre las causas frecuentes de errores de estimación se encuentran:

- Requerimientos o actividades omitidas;
- Diferencias en los supuestos de quienes estiman;
- Desconocimiento técnico o del dominio;
- Olvidar pruebas, integración, documentación u otras actividades necesarias;
- Sobreestimar la disponibilidad real del equipo.

Reuniones, formación, licencias y otras obligaciones también consumen capacidad. Los buffers pueden reservar margen frente a imprevistos, pero **no sustituyen comprender correctamente el trabajo**.

TAMAÑO Y STORY POINTS

En estimación ágil se intenta separar el **tamaño relativo del trabajo** de su duración exacta. No deben confundirse:

- **Tamaño**: magnitud relativa de una historia.
- **Esfuerzo**: trabajo humano necesario.
- **Duración**: tiempo calendario que transcurre.
- **Costo**: consecuencia económica del trabajo y los recursos utilizados.

Estas magnitudes se relacionan, pero **no son equivalentes**.

Los **Story Points (SP)** son una unidad propia del equipo utilizada para expresar el **tamaño relativo** de las historias. Para estimar ese tamaño, se considera principalmente: **esfuerzo relativo + complejidad + duda/incertidumbre**

No existe una fórmula matemática que los sume. El equipo utiliza esas dimensiones para comparar una historia con otras.

Por ejemplo, dos historias podrían recibir el mismo tamaño, aunque una requiera mucho trabajo conocido y la otra menos trabajo, pero mayor complejidad o incertidumbre.

Los Story Points:

- **no son horas ni días;**
- no representan valor de negocio ni prioridad;
- no permiten comparar directamente equipos distintos;
- no deberían convertirse mediante una equivalencia fija del tipo **1 SP = X horas**.

La estimación es relativa porque resulta más sencillo responder: “¿Esta historia es aproximadamente más grande o más pequeña que aquella?” que intentar predecir directamente una cantidad exacta de horas.

HISTORIA CANÓNICA Y ESCALA

Para realizar comparaciones se establece una **User Story canónica**, que funciona como referencia o “metro patrón”. Esta conviene que sea una historia **bien comprendida, de alcance claro y con baja incertidumbre**, de modo que funcione como una referencia estable para comparar las demás.

A la canónica se le asigna un valor y luego las demás historias se estiman relativamente respecto de ella y de otras historias ya estimadas. Esta **no es universal**, una historia que funciona como buena referencia para un equipo o producto puede no servir en otro contexto.

Pueden utilizarse distintas escalas, como talles o secuencias numéricas. En Planning Poker suelen utilizarse valores progresivamente separados, por ejemplo: 1 – 2 – 3 – 5 – 8 – 13 – 20 – 40 – 100

La separación creciente refleja que **cuanto mayor e incierto es el trabajo, menos sentido tiene discutir diferencias pequeñas**. Los valores muy altos también son una señal para preguntarse si la historia debería dividirse. No significa que “13 puntos” tenga un significado universal: depende de la escala y referencia del equipo.

Una vez elegidas la **escala y la referencia, deben mantenerse suficientemente estables para conservar la comparabilidad**.

PLANNING POKER

El **Planning Poker** es una técnica colaborativa de estimación relativa. Compara historias mediante juicio experto, analogía y discusión entre quienes realizarán el trabajo.

El Product Owner participa aclarando el propósito, alcance y criterios de la historia, pero **la estimación corresponde principalmente a quienes deberán realizar el trabajo**. El procedimiento general es:

1. Acordar la historia canónica y la escala.
2. Presentar la historia que se quiere estimar y aclarar las dudas necesarias.
3. Cada integrante selecciona individualmente una estimación.
4. Todas las estimaciones se revelan simultáneamente.
5. Si existen diferencias, se discuten especialmente los valores más altos y más bajos.
6. Se vuelve a estimar hasta alcanzar un acuerdo razonable.

Revelar las cartas simultáneamente evita que la primera opinión condicione inmediatamente al resto.

La **discusión es más importante que alcanzar rápidamente un número**. Una diferencia entre estimadores puede revelar que alguien consideró una integración, una prueba, una dependencia o un riesgo que los demás no habían tenido en cuenta.

Por eso, **no corresponde simplemente promediar valores diferentes sin comprender por qué difieren**.

Si la incertidumbre sigue siendo demasiado grande puede ser necesario **refinar la historia, dividirla o realizar un spike** antes de estimarla.

VELOCIDAD Y PRONÓSTICO

La **velocidad** representa la cantidad de Story Points correspondientes a historias **completamente terminadas** durante una iteración. Los Story Points de una historia incompleta no se contabilizan parcialmente en la velocidad.

La velocidad observada permite relacionar el tamaño pendiente con la capacidad histórica del equipo y elaborar un pronóstico: **Iteraciones aproximadas = Story Points pendientes / velocidad observada por iteración**

Por ejemplo, si quedan 60 SP y el equipo viene completando aproximadamente 15 SP por Sprint: **60 / 15 = 4 Sprints**

Esto es un **pronóstico**, no una garantía. Tiene sentido mientras el equipo, la escala, el alcance y las condiciones sean razonablemente comparables.

La expresión de la cátedra de que “**la velocidad corrige los errores de estimación**” debe entenderse como calibración mediante evidencia real. Si el equipo tiende sistemáticamente a asignar tamaños mayores o menores, la velocidad observada permite igualmente relacionar esos tamaños con lo que efectivamente logra completar. No significa que la velocidad vuelva correcta cualquier estimación.

CUÁNDO REESTIMAR

Los Story Points expresan **tamaño relativo**, no el tiempo que finalmente demoró una historia.

Por eso, una historia terminada **no debe reestimarse simplemente porque demoró más o menos de lo esperado**. Cambiar sus puntos para hacerlos coincidir con el tiempo real distorsionaría la referencia histórica y la velocidad.

Corresponde revisar una estimación cuando aparece nueva información que modifica lo que se entiende sobre el **tamaño relativo del trabajo**, por ejemplo:

- Se descubre funcionalidad que no estaba contemplada;
- Aumenta o disminuye el alcance;
- Aparece una dependencia importante;
- Una incertidumbre técnica resulta mucho mayor de lo previsto.

En cambio, si la historia demoró más porque hubo reuniones, interrupciones, ausencias o menor disponibilidad, **no cambió necesariamente su tamaño**. Cambió la capacidad efectiva del equipo, algo que terminará reflejándose en su velocidad.

La pregunta central es:

¿cambió nuestra comprensión del trabajo o sólo cambió el tiempo/capacidad con que pudimos realizarlo?

STORY POINTS Y TIEMPO

También pueden utilizarse unidades de tiempo para otros propósitos.

Los **días ideales** representan cuánto tardaría una tarea si pudiera trabajarse sin interrupciones ni otras obligaciones. Por eso, un día ideal no equivale necesariamente a un día calendario.

Los Story Points evitan depender directamente de esas diferencias individuales y mantienen el foco en **comparar el tamaño del trabajo desde la perspectiva del equipo**.

Esto no significa que nunca puedan utilizarse horas. Por ejemplo, durante una planificación detallada puede ser útil pensar tareas concretas en tiempo.

Lo que debe evitarse es convertir los Story Points mediante una equivalencia fija: **SP ≠ horas** porque eso elimina la separación entre tamaño relativo y duración que se buscaba conseguir.

SCRUM

Scrum es un framework empírico liviano para generar valor mediante soluciones adaptativas a problemas complejos. Define responsabilidades, eventos, artefactos y compromisos, pero no prescribe todas las prácticas técnicas o de gestión que debe utilizar el equipo.

Scrum se apoya en el **control empírico** trabajado anteriormente:

- **Transparencia:** El trabajo y su estado deben poder observarse de manera confiable.
- **Inspección:** Se analizan frecuentemente resultados y progreso.
- **Adaptación:** Cuando la evidencia muestra diferencias o problemas se modifica lo necesario.

Por eso, Scrum no consiste simplemente en realizar reuniones periódicas. Si la información no representa la realidad, no puede inspeccionarse correctamente; y si se detecta un problema, pero no se adapta nada, el ciclo empírico queda incompleto.

El trabajo se organiza de forma iterativa mediante **Sprints**, lo que permite volver frecuentemente sobre decisiones usando nueva evidencia.

SCRUM 2020 Y AMPLIACIÓN 2026

La cátedra utiliza como base la **Scrum Guide 2020** y agrega conceptos de la **Scrum Guide Expanded (SGEP)**. La ampliación 2026 incorpora, entre otras ideas:

- Product Developers humanos y apoyo automatizado/IA;
- Supporters;
- ciclo OODA;
- una perspectiva explícita de producto;
- Distinción entre **Output Done** y **Outcome Done**.

VALORES Y APRENDIZAJE

Scrum propone cinco valores:

1. **Compromiso:** Trabajar hacia objetivos compartidos.
2. **Foco:** Concentrarse en lo más importante.
3. **Apertura:** Mostrar honestamente avances, problemas y nueva información.
4. **Respeto:** Reconocer capacidades y perspectivas de los demás.
5. **Coraje:** Enfrentar problemas, incertidumbre y desacuerdos necesarios.

La ampliación 2026 relaciona estos valores con el ciclo **OODA: Observar → Orientar → Decidir → Actuar**. En este, primero se obtiene evidencia, luego se interpreta según el contexto, se decide qué hacer y finalmente se actúa, produciendo nueva información que vuelve a iniciar el ciclo.

La idea central vuelve a ser la misma del empirismo: **no asumir que el plan inicial contiene todas las respuestas, sino aprender mediante evidencia y adaptación.**

SCRUM TEAM Y RESPONSABILIDADES

El **Scrum Team** es pequeño, multifuncional y autogestionado. Reúne colectivamente las capacidades necesarias para producir resultados utilizables y decide internamente cómo organizar su trabajo.

Ser **multifuncional** no significa que cada integrante deba saber hacer todo, sino que el equipo en conjunto posea las capacidades necesarias sin depender constantemente de silos externos.

Ser **autogestionado** significa que el equipo decide quién hace qué, cuándo y cómo, en lugar de recibir una distribución detallada de tareas desde una jefatura externa.

Dentro del Scrum Team se distinguen tres responsabilidades principales. El Product Owner, Product Developers y Scrum Master.

PRODUCT OWNER

El **Product Owner (PO)** es responsable de **maximizar el valor del producto**. Entre sus responsabilidades se encuentran:

- Definir y comunicar el **Product Goal**;
- Ordenar el **Product Backlog**;
- Asegurar que ese trabajo sea visible y comprensible;
- Conectar necesidades y objetivos con las decisiones sobre el producto.

Es **una persona**, no un comité. Puede recibir información de muchos interesados e incluso delegar tareas de gestión, pero conserva la responsabilidad final sobre esas decisiones.

El PO se concentra principalmente en **qué se busca y por qué aporta valor**. No determina unilateralmente cómo debe implementarse técnicamente.

PRODUCT DEVELOPERS

Los **Product Developers** crean incrementos utilizables y de calidad. Son responsables de:

- Construir el incremento;
- Planificar y adaptar el Sprint Backlog;
- Estimar el trabajo que realizarán;
- Respetar el estándar de calidad;
- Modificar su plan cuando aparece nueva información, manteniendo el Sprint Goal.

En la ampliación 2026 pueden participar herramientas automatizadas o IA apoyando análisis, generación y otras tareas. Sin embargo, se mantiene **human in the loop**, es decir, la responsabilidad final sobre decisiones, calidad y uso ético continúa siendo humana. Automatizar una actividad no demuestra automáticamente que el resultado tenga calidad o genere valor.

SCRUM MASTER

El **Scrum Master** es responsable de favorecer la **efectividad del Scrum Team y la correcta comprensión y aplicación de Scrum**. Actúa como facilitador, líder al servicio del equipo y agente de cambio. Ayuda a:

- Mejorar la forma de trabajar;
- Favorecer la autogestión;
- Facilitar interacciones;
- Hacer que se eliminen impedimentos;
- Mejorar la adopción de Scrum dentro de la organización.

No es el jefe del equipo, no distribuye tareas, no reemplaza al PO para priorizar y no decide por los Developers cómo implementar.

STAKEHOLDERS Y SUPPORTERS

Los **stakeholders** son personas o entidades interesadas o afectadas por el producto. Pueden aportar necesidades, restricciones y retroalimentación.

La ampliación incorpora al **Supporter**, un stakeholder que contribuye activamente a crear las condiciones necesarias para que Scrum funcione, por ejemplo, eliminando impedimentos organizacionales o facilitando recursos.

El Supporter pertenece al ecosistema ampliado; **no es una cuarta responsabilidad interna equivalente a PO, Product Developers y Scrum Master**.

ARTEFACTOS Y COMPROMISOS

Los artefactos proporcionan **transparencia** sobre el trabajo y sus resultados. Cada uno posee un compromiso asociado que proporciona foco y facilita la inspección.

Artefacto	Qué representa	Compromiso
Product Backlog	Trabajo emergente y ordenado necesario para evolucionar el producto	Product Goal
Sprint Backlog	Sprint Goal + trabajo seleccionado + plan para realizarlo	Sprint Goal
Increment	Trabajo integrado, utilizable y con calidad suficiente	Definition of Done / Output Done
Product	Producto utilizado sobre el que pueden observarse resultados	Definition of Outcome Done

La cuarta relación pertenece a la **ampliación 2026**. La estructura clásica de Scrum mantiene tres artefactos.

PRODUCT BACKLOG, PRODUCT GOAL Y REFINAMIENTO

El **Product Backlog** es una lista **emergente y ordenada** del trabajo relevante para el producto. No es una especificación fija del producto completo. Sus elementos pueden escribirse como User Stories, pero Scrum **no exige** que todos los elementos tengan ese formato.

El **Product Goal** expresa un estado futuro concreto que orienta el trabajo hacia un resultado común.

La **Product Vision** representa una dirección general por lo que es todavía más amplia y de largo plazo.

El **refinamiento** es una actividad continua mediante la cual se descomponen y aclaran elementos, se agregan detalles y criterios de aceptación y se analizan tamaño, dependencias e incertidumbre. El refinamiento no busca especificar anticipadamente todo el producto, sino conseguir claridad suficiente para el trabajo próximo.

SPRINT BACKLOG Y SPRINT GOAL

El **Sprint Goal** expresa el propósito que se busca alcanzar durante el Sprint. Permite que el equipo mantenga un objetivo común sin convertir el conjunto inicial de tareas en algo rígido.

El **Sprint Backlog** contiene: **por qué** → **Sprint Goal**; **qué** → **Product Backlog (Items seleccionados)**; **cómo** → **plan para realizarlos**. Es un plan vivo, donde los Developers pueden modificarlo mientras aprenden siempre que se preserve el Sprint Goal.

INCREMENT

Un **Increment** es trabajo integrado y utilizable que se suma a lo previamente construido. No basta con tener muchas tareas terminadas por separado: las partes deben funcionar conjuntamente y cumplir el estándar de calidad.

Puede existir más de un incremento dentro de un Sprint y un incremento puede liberarse antes de la Sprint Review. Por lo tanto: **Incremento terminado ≠ obligación de esperar a la Review para liberarlo**.

DEFINITION OF OUTPUT DONE

En Scrum 2020 este compromiso se denomina Definition of Done; la ampliación 2026 utiliza Output Done.

Responde principalmente: **¿construimos correctamente una solución utilizable?** Indica el estándar de calidad necesario para considerar que existe un incremento utilizable. Puede incluir:

- integración;
- pruebas;
- seguridad;
- documentación;
- otras condiciones técnicas o de calidad.

DEFINITION OF OUTCOME DONE

Responde: **¿lo construido produjo realmente el resultado que buscábamos?** Define qué evidencia permitirá determinar si el producto produjo el beneficio buscado. Puede observar:

- comportamiento de usuarios;
- resultados del negocio;
- validación de hipótesis;
- otras métricas de valor.

Por eso: **Output Done ≠ Outcome Done**. Un equipo puede construir correctamente una funcionalidad y posteriormente descubrir que casi nadie la utiliza o que no produce la mejora esperada.

El output puede terminarse durante el Sprint; el outcome muchas veces sólo puede observarse **después de que el producto sea utilizado**.

DEFINITION OF READY

La **DoR** indica si existe preparación suficiente para comenzar a trabajar sobre un ítem. Puede considerar:

- Propósito comprendido;
- Criterios de aceptación discutidos;
- Tamaño abordable;
- Dependencias conocidas;
- Información suficiente para comenzar.

No exige certeza completa ni impide aprender durante el Sprint. A diferencia de Done y de los compromisos asociados a artefactos, **Ready surge del refinamiento y no es un compromiso formal equivalente dentro de Scrum**.

EL SPRINT Y LOS TIMEBOXES

El **Sprint** es el evento contenedor dentro del cual se desarrolla el trabajo y ocurren los demás eventos de Scrum. Durante este se construye calidad; no debería planificarse sistemáticamente un Sprint posterior dedicado únicamente a “terminar”, integrar o estabilizar trabajo que debería haber quedado Done.

Los **timeboxes representan límites máximos**, no una obligación de consumir todo el tiempo disponible.

Evento	Timebox
Sprint	Un mes o menos; la ampliación también presenta máximo cuatro semanas
Sprint Planning	Hasta 8 h para un Sprint de un mes
Daily Scrum	15 min
Sprint Review	Hasta 4 h para un Sprint de un mes
Sprint Retrospective	Hasta 3 h para un Sprint de un mes

Para Sprints más cortos, Planning, Review y Retrospective suelen ser más breves.

La presentación también menciona aproximadamente un **10 % para refinamiento**. Conviene interpretarlo como una orientación de ese material: el refinamiento es continuo y ese porcentaje no constituye una regla formal universal.

Si el Sprint Goal deja de tener sentido, el Product Owner puede cancelar el Sprint.

EVENTOS DE SCRUM

Los eventos crean oportunidades regulares para **inspeccionar y adaptar** distintos aspectos del trabajo.

SPRINT PLANNING

Inicia el Sprint y construye un plan compartido. Responde tres preguntas:

¿Por qué este Sprint es valioso? → Sprint Goal.

¿Qué puede realizarse? → selección de Product Backlog Items.

¿Cómo se realizará? → plan de trabajo.

Los Product Developers determinan cuánto pueden realizar considerando prioridades, desempeño previo, capacidad disponible, dependencias y calidad.

La selección representa un **pronóstico**, no una garantía rígida de completar exactamente todos los ítems inicialmente seleccionados. El compromiso principal es mantener el **Sprint Goal y la calidad**, adaptando el plan cuando sea necesario.

DAILY SCRUM

Es una inspección diaria de quince minutos realizada por los Product Developers. Su objetivo es comprobar el avance hacia el **Sprint Goal** y adaptar el plan para continuar. No es:

- Un reporte al Scrum Master;
- Una reunión para justificar cuánto trabajó cada persona;
- Una obligación de utilizar una determinada secuencia de preguntas.

Debe permitir salir con una mejor decisión sobre cómo continuar.

SPRINT REVIEW

La **Sprint Review** inspecciona el producto y la situación actual junto con stakeholders. Se analizan:

- incremento producido;
- avance hacia objetivos;
- cambios en el contexto;
- retroalimentación;
- evidencia de outcomes cuando ya está disponible.

El objetivo es **aprender y adaptar el producto y el Product Backlog**. No es solamente una demostración ni una ceremonia para obtener aprobación.

SPRINT RETROSPECTIVE

La **Retrospective** inspecciona **cómo está trabajando el equipo**. Puede analizar:

- interacciones;
- procesos;
- herramientas;
- prácticas técnicas;
- calidad;
- problemas y oportunidades de mejora.

El equipo selecciona mejoras realizables para aumentar su efectividad.

La diferencia central es:

Review → inspecciona principalmente producto, valor y próximos pasos.

Retrospective → inspecciona principalmente la forma de trabajar.

PLANIFICACIÓN EN VARIOS NIVELES

La planificación ágil no significa ausencia de planificación. La cátedra distingue distintos horizontes:

Estrategia → Portfolio → Producto → Release → Sprint → Día

A medida que el horizonte se acerca, aumenta razonablemente el nivel de detalle.

No tiene sentido describir tareas que podrían realizarse dentro de varios meses con el mismo detalle con que se organiza el trabajo de mañana, porque el aprendizaje probablemente modifique esas decisiones.

También pueden utilizarse diferentes unidades según el nivel:

- Tamaños generales para iniciativas lejanas;
- Story Points para historias;
- Horas u otra unidad detallada para tareas próximas cuando sea útil.

Una **release** es una liberación del producto y un **Sprint** es un período de trabajo. Por eso: **Sprint ≠ Release**. Puede existir una liberación después de varios Sprints, al final de uno o incluso durante un Sprint cuando existe un incremento adecuado para liberar.

CAPACIDAD Y VELOCIDAD

La **capacidad** representa el tiempo efectivo disponible para trabajar durante un Sprint. Para calcularla deben considerarse:

- Disponibilidad de las personas;
- Ausencias;
- Eventos de Scrum;
- Soporte y mantenimiento;
- Trabajo en otros proyectos;
- Una reserva razonable ante incertidumbre.

La **velocidad**, en cambio, representa el tamaño en Story Points que el equipo realmente consigue terminar.

capacidad → disponibilidad de tiempo

velocidad → cantidad relativa de trabajo terminado

Una disminución de capacidad puede reducir lo que el equipo probablemente completará, aunque el tamaño estimado de las historias no haya cambiado.

TRANSPARENCIA Y SEGUIMIENTO

Un **taskboard** hace visible el estado del trabajo y permite detectar tareas bloqueadas o acumulación de trabajo iniciado. El tablero aporta transparencia, pero **mover una tarjeta a “Done” no demuestra por sí mismo calidad ni valor**. Debe existir coherencia con la Definition of Done y con el estado real del producto.

La idea es priorizar terminar antes de iniciar continuamente nuevo trabajo para así producir **incrementos utilizables**, no simplemente mantener a todas las personas ocupadas.

MÉTRICAS Y GRÁFICOS

Las métricas deben proporcionar evidencia útil para tomar decisiones, evitando medir por medir.

BURNDOWN

El **Burndown** representa cuánto trabajo queda pendiente a medida que avanza el tiempo. Permite observar tendencias, pero no explica automáticamente sus causas. Una curva estancada puede indicar bloqueos, historias grandes u otros problemas que deben investigarse. Estar por encima o por debajo de una línea ideal es una **señal para inspeccionar**, no un diagnóstico completo.

BURNUP

El **Burnup** representa trabajo completado acumulado y puede mostrar además el alcance total. Al separar: **trabajo completado** de **alcance total** permite observar mejor si un cambio en el pronóstico se debe a menor progreso o a que aumentó el alcance.

VELOCIDAD Y RTF

La **velocidad** contabiliza Story Points terminados y permite realizar pronósticos del mismo equipo.

Running Tested Features (RTF) contabiliza funcionalidades que funcionan y han sido probadas, priorizando evidencia de funcionalidad utilizable frente a trabajo solamente iniciado.

Ninguna de estas métricas demuestra por sí sola que exista **valor de negocio**. Por eso, el enfoque ampliado agrega la observación de outcomes:

¿terminamos trabajo? → output

¿lo terminado produjo el cambio esperado? → outcome

FUNCIONAMIENTO INTEGRADO DE SCRUM

Scrum debe comprenderse como un ciclo conectado:

Product Vision / Product Goal

↓

Product Backlog ordenado y refinado

↓

Sprint Planning → Sprint Goal + Sprint Backlog

↓

desarrollo e inspección diaria

↓

Increment con calidad / Output Done

↓

uso y evidencia de Outcome

↓

Sprint Review → adaptación del producto

↓

Retrospective → adaptación de la forma de trabajar

↓

nueva información modifica decisiones futuras

El **discovery** —investigar qué problema o solución conviene— y el **delivery** —construir y entregar— no deben entenderse como dos grandes fases completamente separadas. Durante la evolución del producto aparecen continuamente preguntas que requieren aprendizaje mientras se mantiene la capacidad de producir incrementos utilizables.

U3: Gestión de configuración de software

FUNDAMENTOS Y CONCEPTOS DE SCM

La **Software Configuration Management (SCM)** es una disciplina técnica y administrativa cuyo objetivo es **mantener la integridad de los productos de software a medida que evolucionan**.

El software cambia continuamente por nuevos requerimientos, correcciones, mejoras, cambios del negocio, dependencias o decisiones del proyecto. El problema no es impedir esos cambios, sino poder responder:

¿qué elementos forman actualmente el producto? → ¿qué versiones corresponden entre sí? → ¿qué cambió? → ¿quién lo cambió? → ¿qué fue aprobado y probado?

SCM permite conservar esa información y controlar la evolución del producto durante todo su ciclo de vida.

No se limita a Git ni al versionado del código. Puede involucrar requerimientos, diseño, código, pruebas, documentación, ejecutables, scripts, decisiones de cambio y otros artefactos del producto y del proyecto.

INTEGRIDAD DE LA CONFIGURACIÓN

Mantener la **integridad** significa poder identificar qué constituye el producto en un momento determinado y conservar la coherencia entre sus componentes.

SCM busca evitar problemas como:

- Pérdida de componentes o modificaciones;
- Superposición de cambios;
- Falta de sincronía entre código fuente, objetos y ejecutables;
- Reparación de defectos ya corregidos;
- Mantenimiento duplicado;
- Cambios que nunca fueron validados.

Por eso, el problema no es solamente “tener los archivos”. Una organización puede conservar todos los archivos y aun así construir o entregar **una combinación incorrecta de versiones que nunca fue probada conjuntamente**.

ÍTEM DE CONFIGURACIÓN

Un **Ítem de Configuración —IC o SCI—** es un elemento del producto o proyecto cuya identidad y evolución necesitan gestionarse. Puede ser, por ejemplo:

- un requerimiento;
- un modelo o diseño;
- código fuente;
- un ejecutable;
- una prueba;
- un manual;
- un script de construcción o instalación;
- un plan o cronograma;
- un reporte de defecto o solicitud de cambio.

No significa que absolutamente todo artefacto deba recibir el mismo nivel de control. La decisión de convertir algo en IC depende de **cuánto importa conocer su estado, historia y relación con el resto del producto**.

Controlar únicamente el código puede ser insuficiente. Para reproducir o comprender un producto también puede ser necesario conocer qué requerimientos satisface, qué pruebas corresponden, qué dependencias utilizó y cómo debe construirse o instalarse.

VERSIÓN, VARIANTE Y CONFIGURACIÓN

Estos conceptos describen cosas diferentes.

VERSIÓN

Una **versión** representa un estado determinado de un ítem a lo largo de su evolución. Las versiones suelen representar una **evolución temporal** de un mismo elemento.

El control de versiones permite:

- conservar estados anteriores;
- comparar modificaciones;
- reconocer quién cambió qué;
- recuperar una versión específica.

VARIANTE

Una **variante** representa una alternativa que puede evolucionar y coexistir con otras. A diferencia de las versiones, las variantes **no tienen por qué reemplazarse unas a otras**: pueden evolucionar simultáneamente.

Por ejemplo, podrían existir variantes diferentes para:

- distintas plataformas;
- clientes;
- configuraciones funcionales;
- entornos.

CONFIGURACIÓN

Una **configuración de software** es el conjunto de Ítems de Configuración, con sus versiones concretas, que representan el sistema en un momento determinado.

Por ejemplo: **Requerimientos v2 + Diseño v3 + Código commit X + Manual v2** pueden constituir una configuración válida.

Los **números de versión de los distintos IC no necesitan coincidir**. Lo importante es que esa combinación sea la que corresponde al producto que se quiere identificar o reconstruir. Por eso, obtener simplemente “la última versión de cada archivo” no garantiza reconstruir una configuración válida.

REPOSITORIO

El **repositorio** es el medio donde se conservan los Ítems de Configuración y su historia, junto con la información necesaria para recuperarlos y relacionarlos.

No debe pensarse únicamente como una carpeta: su finalidad es permitir conocer la evolución de los elementos y recuperar estados conocidos.

Puede utilizarse un modelo:

- **Centralizado:** Existe un repositorio principal que concentra la historia compartida.
- **Distribuido:** Los participantes pueden disponer localmente de una copia del repositorio y de su historia.

La diferencia técnica modifica la forma de colaborar y recuperar información, pero **ningún modelo elimina la necesidad de definir reglas de identificación, acceso, respaldo e integración**.

Operaciones como checkout, modificación, check-in o commit son mecanismos concretos de herramientas. El objetivo conceptual de SCM es **conservar estados e integrar cambios sin perder la coherencia del producto**.

LÍNEA BASE — BASELINE

Una **línea base** es una **configuración formalmente revisada y acordada que se establece como referencia** para el trabajo posterior. Una vez establecida, los cambios sobre esa referencia deben realizarse mediante el procedimiento de control definido. Una baseline permite saber:

- qué configuración fue aprobada;
- qué cambió desde ella;
- qué versión debe utilizarse como referencia;
- cómo reconstruir un estado histórico conocido.

No debe confundirse con:

- una copia cualquiera del proyecto;
- el último estado disponible;
- una release;
- el número comercial de versión.

Una herramienta puede utilizar un **tag o etiqueta** para marcar técnicamente una baseline, pero **el tag no es lo que convierte a la configuración en línea base**. El carácter de baseline surge de que fue revisada y acordada. Además, la etiqueta utilizada para identificar una línea base no debe confundirse con la versión del producto.

Cuando posteriormente se aprueban cambios, la baseline anterior **no desaparece ni se modifica retroactivamente**. Se conserva como referencia histórica y pueden establecerse nuevas configuraciones o líneas base.

RAMAS E INTEGRACIÓN

Una **branch o rama** permite que una línea de desarrollo evolucione temporalmente de manera separada, es decir, permitir cambios paralelos sin modificar inmediatamente la línea principal. Puede utilizarse, por ejemplo, para:

- Desarrollar una funcionalidad;
- Realizar una corrección;
- Experimentar;
- Mantener una variante del producto.

Un **merge** integra modificaciones de una línea con otra. Cuando existen cambios incompatibles puede aparecer un **conflicto**. Resolverlo no significa solamente conseguir que la herramienta complete el merge: es necesario decidir qué combinación es conceptualmente correcta y después verificarla.

Por eso, crear ramas facilita el trabajo paralelo, pero **no elimina los problemas de coordinación ni integración**.

RELEASE

Una **release** es una versión o configuración preparada para ser puesta a disposición de sus destinatarios. Debe ser posible identificar claramente:

- qué contiene;
- qué versiones la integran;
- de qué configuración proviene;
- qué controles superó.

Conviene distinguir:

Versión → estado de un elemento.

Configuración → combinación identificada de versiones de varios IC.

Baseline → configuración revisada y establecida como referencia.

Branch → línea de trabajo separada.

Release → configuración preparada para ser liberada.

Una misma configuración puede ser simultáneamente una baseline y posteriormente convertirse en una release, pero **los conceptos no significan lo mismo.**

IDEA CENTRAL DE SCM

SCM no busca evitar que el software cambie, sino que busca que el cambio ocurra de una forma en la que podamos conservar: **identidad → historia → trazabilidad → consistencia → posibilidad de reconstrucción.**

Por eso SCM responde directamente a dos problemas que aparecen durante todo el desarrollo de software:

múltiples personas trabajando sobre elementos relacionados + múltiples versiones evolucionando en el tiempo.

ACTIVIDADES DE SCM Y APLICACIÓN ÁGIL

SCM mantiene la integridad del producto mediante cuatro actividades fundamentales:

Identificar qué se controla → Controlar los cambios → Conocer el estado → Comprobar la consistencia.

Estas actividades no buscan impedir que el software cambie, sino hacer que los cambios puedan realizarse sin perder la identidad, trazabilidad y coherencia de la configuración.

1- IDENTIFICACIÓN DE LA CONFIGURACIÓN

La **identificación** determina **qué elementos estarán bajo control y cómo se los reconocerá.** Implica seleccionar los ítems de Configuración, asignar identificadores y reglas de nombrado, organizar su ubicación y permitir reconocer sus versiones y relaciones.

Una buena identificación debe permitir responder, por ejemplo:

¿qué artefacto es? → ¿a qué producto/proyecto pertenece? → ¿qué versión es? → ¿dónde se encuentra? → ¿qué baseline o release integra?

Por eso, nombres ambiguos como “final”, “final nuevo” o “último” dificultan el control.

Una estructura de carpetas ayuda a organizar, pero no alcanza por sí sola: también debe conocerse qué versiones y relaciones conforman cada configuración.

2- CONTROL DE CAMBIOS

El **control de cambios** busca que una modificación propuesta sobre una configuración sea **analizada antes de incorporarse y quede posteriormente registrada y verificada.**

El flujo conceptual es:

Solicitud de cambio → análisis de impacto → decisión → implementación → verificación → actualización de la configuración y trazabilidad.

El análisis debe considerar no sólo el elemento modificado, sino también dependencias, efectos colaterales, costo, tiempo y consecuencias sobre el resto del producto.

La presentación muestra un **Comité de Control de Cambios —CCC o CCB—** formado por representantes de las áreas afectadas. Su función es evaluar el impacto global y decidir según el procedimiento establecido.

Lo importante no es memorizar una estructura organizativa universal, sino distinguir tres estados: **propuesto ≠ aprobado ≠ implementado ≠ verificado.** Por ejemplo, hacer un commit demuestra que alguien registró una modificación técnica; **no demuestra automáticamente que el cambio estaba autorizado ni que fue correctamente verificado.**

3- REGISTRO E INFORMES DE ESTADO

La gestión de estado conserva información sobre **qué ocurrió con los Ítems de Configuración y los cambios realizados sobre ellos**. Debe permitir responder preguntas como:

¿qué versión está vigente?

¿qué cambio fue solicitado?

¿fue aprobado o rechazado?

¿ya fue implementado y verificado?

¿en qué configuración o release aparece?

El objetivo no es simplemente conservar un historial de archivos, sino mantener **trazabilidad entre solicitud, decisión, implementación, verificación y configuración resultante**.

Gran parte de esta información puede obtenerse automáticamente mediante las herramientas utilizadas por el equipo, reduciendo trabajo manual y riesgo de registros desactualizados.

4- AUDITORÍAS DE CONFIGURACIÓN

Las auditorías comprueban que el producto, la configuración declarada, la documentación y los requerimientos sean consistentes. Se distinguen dos auditorías complementarias.

AUDITORÍA FÍSICA — PCA

Comprueba que lo que se declara como parte de la configuración realmente esté presente y corresponda con la documentación. Busca responder: **¿el producto construido y sus componentes corresponden con la configuración y documentación declaradas?**

Relaciona principalmente: **producto construido ↔ configuración/documentación identificada**

AUDITORÍA FUNCIONAL — FCA

Comprueba que la funcionalidad y el desempeño reales sean consistentes con los requerimientos especificados. Busca responder: **¿el producto construido cumple funcionalmente lo que estaba especificado?**

Relaciona principalmente: **producto construido ↔ requerimientos**

La PCA y la FCA son complementarias. Un producto podría estar perfectamente identificado y documentado, pero no satisfacer los requerimientos; o podría comportarse correctamente mientras sus versiones y documentación son inconsistentes.

La **verificación** pregunta si el producto cumple las especificaciones acordadas; la **validación** considera si satisface adecuadamente la necesidad del usuario. Las auditorías aportan evidencia a estos controles, pero **SCM no es sinónimo de testing**.

PLAN DE GESTIÓN DE CONFIGURACIÓN

El **plan de SCM** establece cómo se aplicarán estas actividades dentro del proyecto. El grado de formalidad puede adaptarse al contexto del proyecto.

Puede definir reglas de nombrado, herramientas, responsabilidades, procedimiento de cambios, formularios o registros y mecanismos de auditoría. Su finalidad es acordar **cómo se ejercerá el control**, no generar documentación por sí misma.

Por eso, instalar Git u otra herramienta no significa haber implementado SCM: todavía deben acordarse qué elementos se controlan, cómo se identifican, quién puede modificar o aprobar, cómo se establecen baselines y cómo se verifican los resultados.

SCM EN ENTORNOS ÁGILES

Agilidad y SCM **no son enfoques opuestos**. Un proceso ágil necesita aceptar cambios frecuentes; SCM permite que esos cambios ocurran **sin perder integridad ni trazabilidad**.

Por eso, en un entorno ágil las funciones de SCM siguen existiendo, aunque su implementación pueda ser más liviana, colaborativa y automatizada.

La presentación incluso plantea como preguntas de adaptación qué ocurre con el Comité de Cambios, las auditorías y los informes de estado en agilidad. La enseñanza no es que deban eliminarse, sino que **la necesidad de control permanece, aunque cambie la forma de ejercerlo**.

SCM se integra al trabajo del equipo: identificación, integración, revisión y automatización contribuyen a producir incrementos consistentes y no deberían dejarse como controles externos realizados únicamente al final.

BUILD, INTEGRACIÓN Y AUTOMATIZACIÓN

Un **build** transforma una configuración conocida de fuentes y otros insumos en componentes o en un producto ejecutable. La posibilidad de repetir un build sobre una configuración identificada ayuda a responder una pregunta esencial de SCM:

¿podemos reconstruir nuevamente el producto que afirmamos haber generado?

La **integración continua (CI)** incorpora y comprueba cambios frecuentemente, permitiendo detectar incompatibilidades antes de que se acumulen.

Automatizar builds y pruebas ayuda a: repetir el proceso de manera consistente → detectar problemas temprano → obtener evidencia frecuente sobre la configuración integrada.

Pero la automatización **no reemplaza las demás actividades de SCM ni demuestra por sí sola que el producto satisface una necesidad**.

También conviene distinguir:

Build → construir el producto.

Integrar → combinar y comprobar cambios.

Desplegar → instalar una versión en un entorno.

Release → poner una versión a disposición de sus destinatarios.

Son actividades relacionadas, pero no equivalentes.

RELACIÓN CON EL RESTO DE LA MATERIA

Los requerimientos e historias expresan **qué se necesita**; el proyecto organiza **cómo llevar adelante el trabajo**; Scrum permite **inspeccionar y adaptar**; y SCM mantiene identificables y coherentes los elementos que cambian durante esa evolución.

Una modificación de una User Story puede afectar requerimientos, diseño, código, pruebas y documentación. SCM permite relacionar esos elementos y conocer **qué configuración implementa y verifica determinada necesidad**.

Esto conecta directamente con la **mutabilidad del software** estudiada anteriormente: **la solución no es impedir el cambio, sino poder introducirlo conservando control e integridad**.

Por eso: **agilidad necesita aprender y cambiar frecuentemente → SCM permite hacerlo sin perder la coherencia del producto**.